

Primer Parcial: Cobro y facturación en gasolinera

Diego Andhres Fernando Barillas Gonzalez, 201602641*

Facultad de Ingeniería, Universidad de San Carlos, Ciudad Universitaria, Zona 12, Guatemala

I. DESCRIPCIÓN GENERAL

Se desea desarrollar un programa en el lenguaje de programación de octave y python consola que permita a una gasolinera llevar un registro de ventas, realizar el cobro y emitir facturas para sus clientes. El sistema debe ser capaz de registrar información sobre el tipo de combustible vendido, la cantidad de litros despachados, el precio por litro, calcular el monto total a pagar, y emitir un recibo de factura. Toda la información registrada debe almacenarse en un archivo de texto y una base de datos PostgreSQL para mantener un historial de transacciones.

II. CÓDIGO UTILIZADO EN PYTHON

```
1 import psycopg2
2
3 # Función para conectar a la base de datos
4 # PostgreSQL
5 def conectar_bd():
6     return psycopg2.connect(
7         user="andhres",
8         password="abc123",
9         host="localhost",
10        port="5432",
11        database="bd_andhres"
12    )
13
14 # Función para crear las tablas si no existen
15 def crear_tablas(conn):
16     with conn.cursor() as cursor:
17         cursor.execute("""
18             CREATE TABLE IF NOT EXISTS
19             menuPrecios (
20                 combustible VARCHAR(50) PRIMARY
21                 KEY,
22                 precio_por_litro NUMERIC(10, 2)
23                 NOT NULL
24             );
25         """)
26     cursor.execute("""
27         CREATE TABLE IF NOT EXISTS
28         menuGasolinera (
```

```
24         id SERIAL PRIMARY KEY,
25         nombre VARCHAR(100) NOT NULL,
26         identificacion VARCHAR(50) NOT
27         NULL,
28         combustible VARCHAR(50) NOT NULL
29         REFERENCES menuPrecios(
30             combustible),
31         litros NUMERIC(10, 2) NOT NULL,
32         monto_total NUMERIC(10, 2) NOT
33         NULL
34     );
35     """
36     conn.commit()
37
38 # Función para actualizar los precios de los
39 # combustibles en la base de datos
40 def inicializar_precios_combustibles(conn):
41     precios = {
42         'Regular': 36.90,
43         'Premium': 28.80,
44         'Diesel': 30.50
45     }
46
47     with conn.cursor() as cursor:
48         for combustible, precio in precios.
49         items():
50             cursor.execute(
51                 """
52                 INSERT INTO menuPrecios (
53                     combustible,
54                     precio_por_litro)
55                 VALUES (%s, %s)
56                 ON CONFLICT (combustible) DO
57                 UPDATE SET precio_por_litro
58                 = EXCLUDED.precio_por_litro;
59                 """
60                 ,
61                 (combustible, precio)
62             )
63     conn.commit()
64
65 # Función para obtener los precios de todos
66 # los combustibles
67 def obtener_precios_combustibles(conn):
68     with conn.cursor() as cursor:
69         cursor.execute("SELECT combustible,
70             precio_por_litro FROM menuPrecios")
71         precios = cursor.fetchall()
72         return {combustible: float(precio) for
73             combustible, precio in precios}
74
75 # Función para obtener el precio de un tipo de
76 # combustible desde la tabla menuPrecios
77 def obtener_precio_combustible(conn,
78     tipo_combustible):
79     with conn.cursor() as cursor:
```

* 3003241380101@ingenieria.usac.edu.gt

```

63     cursor.execute("SELECT precio_por_litro
        FROM menuPrecios WHERE combustible
        = %s", (tipo_combustible,))
64     resultado = cursor.fetchone()
65     if resultado:
66         return float(resultado[0])
67     else:
68         print(f"El combustible {
            tipo_combustible} no está
            disponible.")
69         return None
70
71 # Función para guardar los datos del cliente
    en la tabla menuGasolinera y en un archivo
    de texto
72 def guardar_datos_cliente(conn, nombre,
    identificacion, combustible, litros,
    monto_total):
73     with conn.cursor() as cursor:
74         cursor.execute(
75             "INSERT INTO menuGasolinera (nombre
                , identificacion, combustible,
                litros, monto_total) VALUES (%s
                , %s, %s, %s, %s)",
76             (nombre, identificacion,
                combustible, litros,
                monto_total)
77         )
78     conn.commit()
79
80 # Obtener el precio por litro del
    combustible seleccionado
81 precio_por_litro =
    obtener_precio_combustible(conn,
    combustible)
82 if precio_por_litro is None:
83     return # Si no se encuentra el precio,
        no se guarda en el archivo
84
85 # Guardar en archivo de texto
86 with open("C:\\Users\\BEST COMPUTER\\
    Desktop\\ProyectosIE\\PrimerParcial\\
    facturas.txt", "a") as archivo:
87     archivo.write(f"NOMBRE DEL CLIENTE: {
        nombre}\n")
88     archivo.write(f"IDENTIFICACIÓN DEL VEHÍ
        CULO: {identificacion}\n")
89     archivo.write(f"TIPO DE COMBUSTIBLE: {
        combustible}\n")
90     archivo.write(f"CANTIDAD DE LITROS: {
        litros}\n")
91     archivo.write(f"PRECIO POR LITRO: Q{
        precio_por_litro:.2f}\n")
92     archivo.write(f"MONTO TOTAL A PAGAR: Q{
        monto_total:.2f}\n")
93     archivo.write("\n") # Añadir una línea
        en blanco
94     archivo.write("*****\n") #
        Línea de separación
95     archivo.write("\n") # Línea en blanco
        para mayor separación

```

```

96
97 # Función para mostrar todos los registros
    ingresados
98 def mostrar_registros(conn):
99     with conn.cursor() as cursor:
100         cursor.execute("SELECT * FROM
            menuGasolinera")
101         registros = cursor.fetchall()
102         if registros:
103             print("\nRegistros en la base de
                datos:")
104             for registro in registros:
105                 print(f"ID: {registro[0]},
                    Nombre: {registro[1]},
                    Identificación: {registro
                        [2]}, Combustible: {registro
                            [3]}, Litros: {registro[4]},
                    Monto Total: Q{registro
                        [5]:.2f}")
106             else:
107                 print("No hay registros en la base
                    de datos.")
108
109 # Función para eliminar un registro mediante
    identificación
110 def eliminar_registro(conn, identificacion):
111     with conn.cursor() as cursor:
112         cursor.execute("DELETE FROM
            menuGasolinera WHERE identificacion
            = %s", (identificacion,))
113         conn.commit()
114         if cursor.rowcount > 0:
115             print(f"Registro con identificación
                {identificacion} eliminado
                exitosamente.")
116         else:
117             print(f"No se encontró ningún
                registro con identificación {
                    identificacion}.")
118
119 def main():
120     # Conexión a la base de datos
121     conn = conectar_bd()
122
123     # Crear las tablas si no existen
124     crear_tablas(conn)
125
126     # Ingresar los precios de los combustibles
127     inicializar_precios_combustibles(conn)
128
129     while True:
130         # Solicitar datos del usuario
131         nombre = input("Ingrese su nombre: ")
132         identificacion = input("Ingrese su nú
            mero de placa: ")
133
134         # Obtener precios actualizados
135         precios_combustibles =
            obtener_precios_combustibles(conn)
136
137         # Menú de combustibles

```

```

138 print("Elija el tipo de combustible que
      desea.")
139 for i, (combustible, precio) in
      enumerate(precios_combustibles.
      items(), start=1):
140     print(f"{i}. {combustible} - Q{
      precio:.2f}")
141
142 opcion_combustible = input("Ingrese el
      número de su opción: ")
143
144 if opcion_combustible == '1':
145     tipo_combustible = 'Regular'
146 elif opcion_combustible == '2':
147     tipo_combustible = 'Premium'
148 elif opcion_combustible == '3':
149     tipo_combustible = 'Diesel'
150 else:
151     print("Opción no válida.")
152     continue
153
154 # Obtener el precio del combustible
      elegido
155 precio_por_litro =
      obtener_precio_combustible(conn,
      tipo_combustible)
156 if precio_por_litro is None:
157     continue
158
159 # Solicitar cantidad de litros
160 try:
161     litros = float(input("Ingrese la
      cantidad en litros a despachar:
      "))
162     if litros <= 0:
163         raise ValueError("La cantidad de
      litros debe ser mayor a
      cero.")
164 except ValueError as e:
165     print(f"Error: {e}")
166     continue
167
168 # Calcular el monto total
169 monto_total = litros * precio_por_litro
170
171 # Mostrar el monto total a pagar
172 print(f"El monto total a pagar por {
      litros} litros de {tipo_combustible
      } es: Q{monto_total:.2f}")
173
174 # Guardar los datos del cliente en la
      base de datos y en un archivo
175 guardar_datos_cliente(conn, nombre,
      identificacion, tipo_combustible,
      litros, monto_total)
176
177 while True:
178     print("\n¿Qué desea hacer a
      continuación?")
179     print("1. Ver todos los registros")

```

```

180 print("2. Continuar con el próximo
      cliente")
181 print("3. Eliminar registro
      mediante identificación")
182 print("4. Salir del programa")
183
184 opcion = input("Ingrese el número
      de su opción: ")
185
186 if opcion == '1':
187     mostrar_registros(conn)
188 elif opcion == '2':
189     break # Salir del bucle interno
      para continuar con el pró
      ximo cliente
190 elif opcion == '3':
191     identificacion_a_eliminar =
      input("Ingrese la placa del
      vehículo cuyo registro desea
      eliminar: ")
192     eliminar_registro(conn,
      identificacion_a_eliminar)
193 elif opcion == '4':
194     conn.close()
195     return # Salir del programa
196 else:
197     print("Opción no válida.")
198
199 if __name__ == "__main__":
200     main()

```

III. CÓDIGO UTILIZADO EN OCTAVE

```

1 pkg load database
2
3 % Función para conectar a la base de datos
      PostgreSQL
4 function conn = conectar_bd()
5     conn = pq_connect(setdbopts('dbname', '
      bd_andhres', 'host', 'localhost', '
      port', '5432', 'user', 'andhres', '
      password', 'abc123'));
6 endfunction
7
8 % Función para crear las tablas si no existen
9 function crear_tablas(conn)
10     pq_exec_params(conn, "CREATE TABLE IF NOT
      EXISTS menuPrecios (combustible
      VARCHAR(50) PRIMARY KEY,
      precio_por_litro NUMERIC(10, 2) NOT
      NULL);");
11     pq_exec_params(conn, "CREATE TABLE IF NOT
      EXISTS menuGasolinera (id SERIAL
      PRIMARY KEY, nombre VARCHAR(100) NOT
      NULL, identificacion VARCHAR(50) NOT
      NULL, combustible VARCHAR(50) NOT NULL
      REFERENCES menuPrecios(combustible),
      litros NUMERIC(10, 2) NOT NULL,
      monto_total NUMERIC(10, 2) NOT NULL);"

```

```

    );
12 endfunction
13
14 % Función para actualizar los precios de los
    combustibles en la base de datos
15 function inicializar_precios_combustibles(conn
    )
16     precios = {'Regular', 36.90; 'Premium',
        28.80; 'Diesel', 30.50};
17     for i = 1:size(precios, 1)
18         pq_exec_params(conn, ["INSERT INTO
            menuPrecios (combustible,
            precio_por_litro) VALUES ($1, $2)
            ON CONFLICT (combustible) DO UPDATE
            SET precio_por_litro = EXCLUDED.
            precio_por_litro;"], precios{i, :})
            ;
19     endfor
20 endfunction
21
22 % Función para obtener los precios de todos
    los combustibles
23 function precios =
    obtener_precios_combustibles(conn)
24     resultados = pq_exec_params(conn, "SELECT
        combustible, precio_por_litro FROM
        menuPrecios;");
25     precios = struct();
26     for i = 1:rows(resultados)
27         precios(resultados{i, 1}) = resultados
            {i, 2};
28     endfor
29 endfunction
30
31 % Función para obtener el precio de un tipo de
    combustible desde la tabla menuPrecios
32 function precio = obtenerPrecioCombustible(
    conn, tipo_combustible)
33     resultados = pq_exec_params(conn, "SELECT
        precio_por_litro FROM menuPrecios
        WHERE combustible = $1; ", {
            tipo_combustible});
34     if isempty(resultados)
35         disp(["El combustible ",
            tipo_combustible, " no está
            disponible."]);
36         precio = NaN;
37     else
38         precio = resultados{1, 1};
39     endif
40 endfunction
41
42 % Función para guardar los datos del cliente
    en la tabla menuGasolinera y en un archivo
    de texto
43 function guardar_datos_cliente(conn, nombre,
    identificacion, combustible, litros,
    monto_total)
44     pq_exec_params(conn, "INSERT INTO
        menuGasolinera (nombre, identificacion
        , combustible, litros, monto_total)

```

```

        VALUES ($1, $2, $3, $4, $5);", {nombre
        , identificacion, combustible, litros,
        monto_total});
45
46     precio_por_litro =
        obtenerPrecioCombustible(conn,
        combustible);
47     if isnan(precio_por_litro)
48         return;
49     endif
50
51     % Guardar en archivo de texto
52     archivo = fopen("facturas.txt", "a");
53     fprintf(archivo, "NOMBRE DEL CLIENTE: %s\n"
        , nombre);
54     fprintf(archivo, "IDENTIFICACIÓN DEL VEHÍ
        CULO: %s\n", identificacion);
55     fprintf(archivo, "TIPO DE COMBUSTIBLE: %s\n"
        , combustible);
56     fprintf(archivo, "CANTIDAD DE LITROS: %.2f\
        n", litros);
57     fprintf(archivo, "PRECIO POR LITRO: Q%.2f\n"
        , precio_por_litro);
58     fprintf(archivo, "MONTO TOTAL A PAGAR: Q%.2
        f\n", monto_total);
59     fprintf(archivo, "\n
        *****\
        n\n");
60     fclose(archivo);
61 endfunction
62
63 % Función para mostrar todos los registros
    ingresados
64 function mostrar_registros(conn)
65     resultados = pq_exec_params(conn, "SELECT *
        FROM menuGasolinera;");
66     if isempty(resultados)
67         disp("No hay registros en la base de
            datos.");
68     else
69         disp("\nRegistros en la base de datos:"
            );
70         for i = 1:rows(resultados)
71             fprintf("ID: %d, Nombre: %s,
                Identificación: %s, Combustible
                : %s, Litros: %.2f, Monto Total
                : Q%.2f\n", resultados{i, 1},
                resultados{i, 2}, resultados{i,
                3}, resultados{i, 4},
                resultados{i, 5}, resultados{i,
                6});
72         endfor
73     endif
74 endfunction
75
76 % Función para eliminar un registro mediante
    identificación
77 function eliminar_registro(conn,
    identificacion)
78     pq_exec_params(conn, "DELETE FROM
        menuGasolinera WHERE identificacion =

```

```

    $1;", {identificacion});
79 if pq_affected_rows(conn) > 0
80     disp(["Registro con identificación ",
            identificacion, " eliminado
            exitosamente."]);
81 else
82     disp(["No se encontró ningún registro
            con identificación ",
            identificacion, "."]);
83 endif
84 endfunction
85
86 % Función principal
87 function main()
88     % Conexión a la base de datos
89     conn = conectar_bd();
90
91     % Crear las tablas si no existen
92     crear_tablas(conn);
93
94     % Ingresar los precios de los combustibles
95     inicializar_precios_combustibles(conn);
96
97     while true
98         % Solicitar datos del usuario
99         nombre = input("Ingrese su nombre: ", "
            s");
100         identificacion = input("Ingrese su nú
            mero de placa: ", "s");
101
102         % Obtener precios actualizados
103         precios_combustibles =
            obtener_precios_combustibles(conn);
104
105         % Menú de combustibles
106         disp("Elija el tipo de combustible que
            desea.");
107         combustibles = fieldnames(
            precios_combustibles);
108         for i = 1:length(combustibles)
109             fprintf("%d. %s - Q%.2f\n", i,
                combustibles{i},
                precios_combustibles.(
                    combustibles{i}));
110         endfor
111
112         opcion_combustible = input("Ingrese el
            número de su opción: ");
113
114         if opcion_combustible > 0 &&
            opcion_combustible <= length(
                combustibles)
115             tipo_combustible = combustibles{
                opcion_combustible};
116         else
117             disp("Opción no válida.");
118             continue;
119         endif
120
121         % Obtener el precio del combustible
            precio_por_litro =
            obtener_precio_combustible(conn,
                tipo_combustible);
122         if isnan(precio_por_litro)
123             continue;
124         endif
125
126         % Solicitar cantidad de litros
127         litros = input("Ingrese la cantidad en
            litros a despachar: ");
128         if litros <= 0
129             disp("La cantidad de litros debe
                ser mayor a cero.");
130             continue;
131         endif
132
133         % Calcular el monto total
134         monto_total = litros * precio_por_litro
            ;
135
136         % Mostrar el monto total a pagar
137         fprintf("El monto total a pagar por %.2
            f litros de %s es: Q%.2f\n", litros
            , tipo_combustible, monto_total);
138
139         % Guardar los datos del cliente en la
            base de datos y en un archivo
140         guardar_datos_cliente(conn, nombre,
            identificacion, tipo_combustible,
            litros, monto_total);
141
142         while true
143             disp("\n¿Qué desea hacer a
                continuación?");
144             disp("1. Ver todos los registros");
145             disp("2. Continuar con el próximo
                cliente");
146             disp("3. Eliminar registro mediante
                identificación");
147             disp("4. Salir del programa");
148
149             opcion = input("Ingrese el número
                de su opción: ");
150
151             if opcion == 1
152                 mostrar_registros(conn);
153             elseif opcion == 2
154                 break; % Salir del bucle interno
                    para continuar con el pró
                    ximo cliente
155             elseif opcion == 3
156                 identificacion_a_eliminar =
                    input("Ingrese la placa del
                        vehículo cuyo registro desea
                        eliminar: ", "s");
157                 eliminar_registro(conn,
                    identificacion_a_eliminar);
158             elseif opcion == 4
159                 pq_close(conn);
160                 disp("Conexión con PostgreSQL
                    cerrada.");
161

```

```

162         return; % Salir del programa
163     else
164         disp("Opción no válida.");
165     endif
166 endwhile
167 endwhile
168 endfunction
169
170 % Ejecutar el programa
171 main();

```

IV. DIAGRAMA DE FLUJO

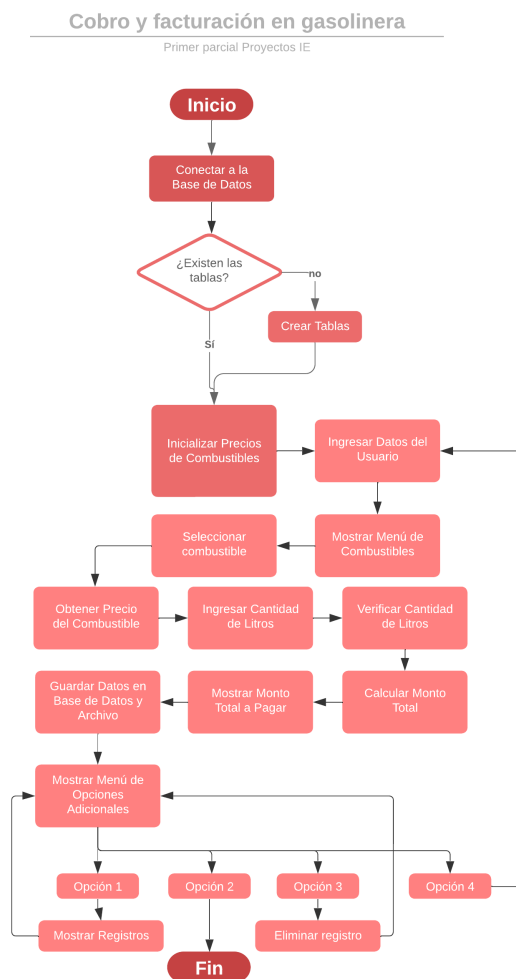


FIG. 1. Diagrama de flujo

V. ALGORITMO:

1. Inicio
2. Conectar a la Base de Datos

- Establecer conexión con la base de datos PostgreSQL usando las credenciales proporcionadas.

3. Crear Tablas si No Existen

- Ejecutar comandos SQL para crear las tablas `menuPrecios` y `menuGasolinera` si no existen.

4. Inicializar Precios de Combustibles

- Definir un diccionario con los precios iniciales de los combustibles.
- Insertar o actualizar los precios en la tabla `menuPrecios`.

5. Mientras (Verdadero)

- Ingresar Datos del Usuario
 - Solicitar al usuario su nombre y número de placa.
- Mostrar Menú de Combustibles
 - Obtener los precios actualizados de los combustibles de la base de datos.
 - Mostrar un menú con las opciones de combustibles disponibles.
- Seleccionar Combustible
 - Solicitar al usuario seleccionar un tipo de combustible
 - Si la opción no es válida, mostrar un mensaje de error y volver a mostrar el menú de combustibles.
- Obtener Precio del Combustible Seleccionado
 - Obtener el precio del combustible elegido desde la base de datos

- Ingresar Cantidad de Litros
 - Solicitar al usuario ingresar la cantidad de litros.
 - Si la cantidad de litros no es válida (negativa), mostrar un mensaje de error y volver a solicitar la cantidad.

- Calcular Monto Total
 - Calcular el monto total a pagar multiplicando la cantidad de litros por el precio del litro.
- Mostrar Monto Total a Pagar
 - Mostrar el monto total a pagar por el usuario
- Guardar Datos en Base de Datos y Archivo
 - Insertar los datos del cliente en la tabla `menuGasolinera`.
 - Guardar los datos del cliente en el archivo de texto `facturas.txt` en el formato especificado, con un separador entre registros.
- Mostrar Menú de Opciones Adicionales
 - Mostrar opciones para ver todos los registros, continuar con el próximo cliente, eliminar un registro, o salir del programa.
- Opciones del Menú Adicional
 - Opción 1: Ver Todos los Registros
 - * Mostrar todos los registros de la tabla `menuGasolinera`.
 - Opción 2: Continuar con el Próximo Cliente
 - * Volver a solicitar los datos del usuario.
 - Opción 3: Eliminar Registro Mediante Identificación
 - * Solicitar la identificación del registro a eliminar.
 - * Eliminar el registro correspondiente de la tabla `menuGasolinera`.
 - Opción 4: Salir del Programa

* Cerrar la conexión a la base de datos y terminar el programa.

6. Cerrar Conexión y Terminar

7. Fin

VI. ALGORITMO EN PSEUDOCÓDIGO:

```

1 Inicio
2   Conectar a la Base de Datos
3   Crear Tablas si No Existen
4   Inicializar Precios de Combustibles
5
6   Mientras Verdadero
7     Ingresar Datos del Usuario
8
9     Mostrar Menú de Combustibles
10    Seleccionar Combustible
11    Si Opción no válida
12      Mostrar mensaje de error
13      Volver a Mostrar Menú de Combustibles
14    FinSi
15
16    Obtener Precio del Combustible Seleccionado
17    Ingresar Cantidad de Litros
18    Si Cantidad de Litros no válida
19      Mostrar mensaje de error
20      Volver a Ingresar Cantidad de Litros
21    FinSi
22
23    Calcular Monto Total
24    Mostrar Monto Total a Pagar
25    Guardar Datos en Base de Datos y Archivo
26
27    Mostrar Menú de Opciones Adicionales
28    Opción = Solicitar opción del menú
29
30    Según Opción
31      Caso 1:
32        Mostrar Todos los Registros
33      Caso 2:
34        Continuar con el Próximo Cliente
35      Caso 3:
36        Eliminar Registro Mediante Identificación
37      Caso 4:
38        Salir del Programa
39        Cerrar Conexión y Terminar
40        Salir
41    Fin Según
42  Fin Mientras
43
44  Cerrar Conexión y Terminar
45 Fin

```